RAT IN A MAZE

A PROJECT REPORT

Submitted by
Vaibhav Kumar
(23BCS13386)

Submitted to:
Mr. Mohammad Shaqlain

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE & ENGINEERING

Chandigarh University

November, 2025

RAT IN A MAZE - PROJECT REPORT

Abstract

The 'Rat in a Maze' problem is a classic and fundamental example of how recursion and backtracking can be used to solve pathfinding problems in computer science. It involves navigating a two-dimensional grid representing a maze, where the rat must move from the starting point (usually the top-left corner) to the destination (bottom-right corner). The movement is constrained to open paths represented by 1s, while 0s represent walls or obstacles. This project explores the problem statement, algorithmic solution, and implementation details using the C++ programming language.

Problem Definition and Motivation

The problem is to determine whether there exists a path for a rat to move from the start position to the destination in a given maze matrix. The rat can only move forward or downward at any point in time. If the rat encounters a blocked cell (represented by 0), it must backtrack and try a different route. This project was chosen because it clearly demonstrates the concept of backtracking—a vital topic in algorithm design and artificial intelligence. By solving this problem, one can understand how recursive calls are used to explore multiple paths efficiently.

Theory of Backtracking

Backtracking is a general algorithmic technique that incrementally builds candidates to solutions and abandons a candidate as soon as it determines that it cannot lead to a valid solution. In the Rat in a Maze problem, the algorithm tries moving in all possible directions recursively. If a move leads to a dead end, it backtracks to the previous step and tries another direction. This approach ensures that every possible path is explored until a valid one is found. It is a depth-first search technique where the path is explored completely before moving to another alternative.

Algorithm and Steps

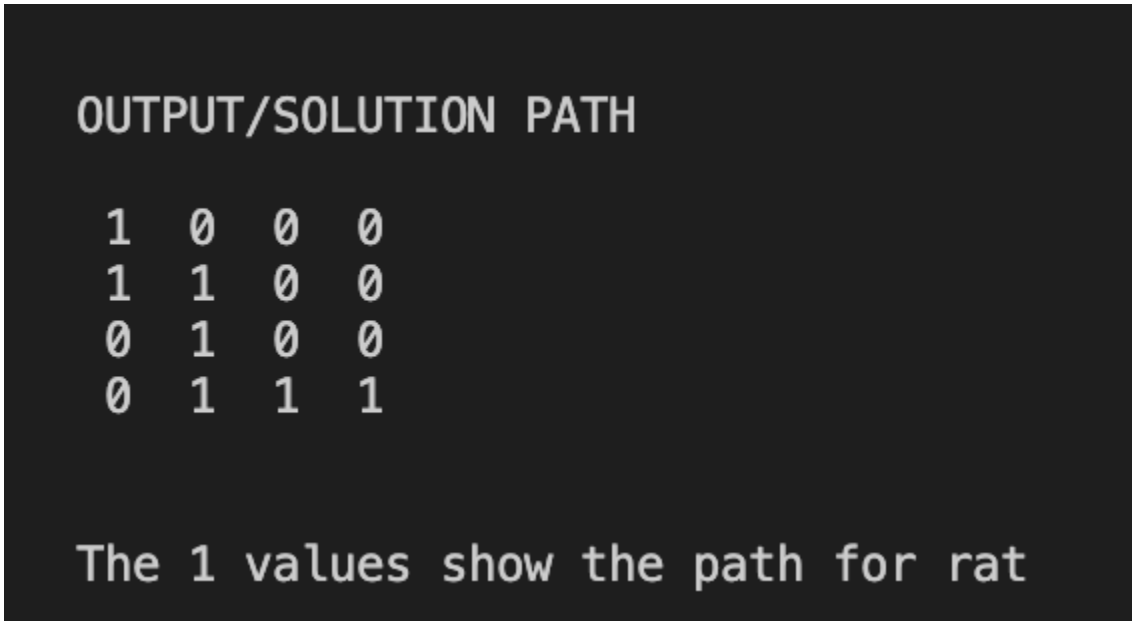
Step 1: Start from the initial cell (0, 0).
Step 2: Check if the current cell is the destination; if yes, print the path and return true.
Step 3: If the cell is valid (inside the maze and not blocked), mark it as part of the solution path.
Step 4: Move to the next cell — either in the x direction (right) or y direction (down).
Step 5: Recursively check if the next move leads to a solution.
Step 6: If no move works, backtrack by unmarking the current cell and return false.
Step 7: Repeat the process until the destination is reached.

C++ Source Code

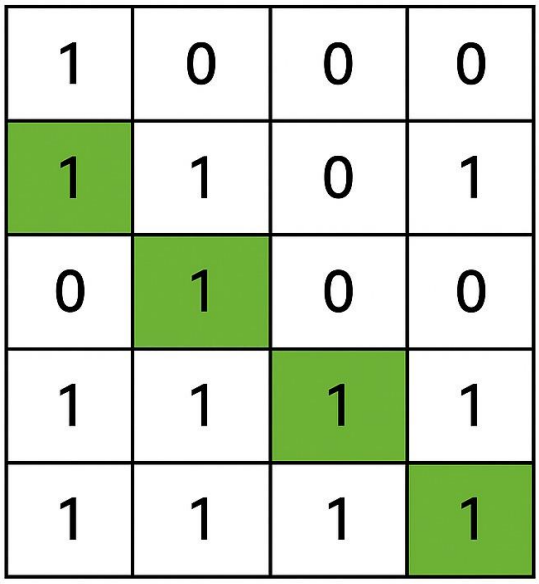
```
#include <stdio.h>
#include <stdbool.h>
#define N 4

bool solveMazeUtil(int maze[N][N], int x, int y, int sol[N][N]);

void printSolution(int sol[N][N]) {
   for (int i = 0; i < N; i++) {
      for (int j = 0; j < N; j++)
         printf(" %d ", sol[i][j]);
      printf("\n");
   }
}

bool isSafe(int maze[N][N], int x, int y) {
   if (x >= 0 && x < N && y >= 0 && y < N && maze[x][y] == 1)
      return true;
   return false;
}

bool solveMaze(int maze[N][N]) {
   int sol[N][N] = {{0, 0, 0, 0},
                    {0, 0, 0, 0},
                    {0, 0, 0, 0},
                    {0, 0, 0, 0}};

   if (solveMazeUtil(maze, 0, 0, sol) == false) {
      printf("Solution doesn't exist");
      return false;
   }
   printSolution(sol);
   return true;
}

bool solveMazeUtil(int maze[N][N], int x, int y, int sol[N][N]) {
   if (x == N - 1 && y == N - 1 && maze[x][y] == 1) {
      sol[x][y] = 1;
      return true;
   }

   if (isSafe(maze, x, y)) {
      if (sol[x][y] == 1)
```

```
            return false;

        sol[x][y] = 1;

        if (solveMazeUtil(maze, x + 1, y, sol))
            return true;

        if (solveMazeUtil(maze, x, y + 1, sol))
            return true;

        sol[x][y] = 0;
        return false;
    }
    return false;
}

int main() {
    int maze[N][N] = {{1, 0, 0, 0},
                      {1, 1, 0, 1},
                      {0, 1, 0, 0},
                      {1, 1, 1, 1}};
    solveMaze(maze);
    return 0;
}
```

Output



OUTPUT/SOLUTION PATH



-

Dry Run Example

Consider the following 4x4 maze:
1 0 0 0
1 1 0 1
0 1 0 0
1 1 1 1

Starting at (0, 0), the rat moves down to (1, 0), then right to (1, 1). From there, it continues down to (2, 1) and then to (3, 1). Next, it moves right through (3, 2) to reach (3, 3), which is the destination. This sequence of moves (Down, Right, Down, Down, Right, Right) represents one valid solution path.

Complexity Analysis

Time Complexity: O(2^(n^2)). The recursive exploration may potentially explore all paths in the worst case.
Space Complexity: O(n^2). Additional matrix storage is used to store the path taken by the rat.
Although not the most optimal solution, it demonstrates clarity and correctness using simple recursion.

Applications

1. Robotics pathfinding and AI-based navigation systems.
2. Game development for generating and solving maze-based levels.
3. Route optimization in logistics and network design.
4. Concept foundation for complex algorithms like N-Queens, Sudoku solving, and graph traversal.

Conclusion

The 'Rat in a Maze' project effectively demonstrates the concept of recursion and backtracking. It provides a structured approach to understanding how algorithms explore and eliminate possibilities to reach valid outcomes. The program can be extended to support larger grids, multiple directions, or graphical visualization to simulate real-world pathfinding problems.

References

1. Introduction to Algorithms by Cormen et al.
2. GeeksforGeeks - Backtracking Problems.
3. TutorialsPoint - Rat in a Maze Explanation.
4. Online Compiler Resources for C++.